



Universidade do Minho
Escola de Engenharia

Computação Gráfica

Trabalho Prático: Fase 2

Licenciatura em Engenharia Informática
Ano Letivo 2025/2026

Grupo 03

A106919 Eduardo Freitas Fernandes
A106842 Gonçalo Rodrigues Ribeiro
A106888 José Mário Raimundo Lima

CG

Índice

1. Introdução	2
2. Arquitetura do Sistema	2
2.1. Classe SceneParser	2
2.2. Hierarquia Transformation	3
2.3. Classe Group	3
3. Transformações Geométricas	4
3.1. Translação	4
3.2. Rotação	5
3.3. Escala	5
4. Demo Scene	5
4.1. Estrutura Hierárquica	6
4.2. Herança de Transformações	6
5. Controlo da Câmara	7
6. Conclusão	7

1. Introdução

O presente relatório descreve o desenvolvimento da segunda fase do projeto prático de Computação Gráfica. Esta fase tem como objetivo principal a extensão do sistema desenvolvido na fase anterior, introduzindo suporte a transformações geométricas hierárquicas e a organização da cena em forma de grafo de cena (*scene graph*).

Relativamente à fase 1, o *Engine* foi reestruturado para suportar grupos aninhados, onde cada grupo pode conter um conjunto de transformações geométricas, modelos e subgrupos. Esta hierarquia permite a criação de cenas complexas com relações de dependência entre objetos, como planetas com luas que herdaram o posicionamento do planeta pai.

Para concretizar esta extensão, foram introduzidas duas novas componentes: a classe `SceneParser`, responsável pelo carregamento e interpretação do ficheiro XML de configuração, e a hierarquia de classes `Transformation`, que abstrai as operações de translação, rotação e escala. A classe `Group` foi igualmente reformulada para acomodar transformações e subgrupos.

A *demo scene* obrigatória desta fase é um modelo estático do sistema solar, incluindo o Sol, os oito planetas e algumas luas, organizados numa hierarquia de grupos que reflete as relações de posicionamento relativo entre os corpos celestes.

2. Arquitetura do Sistema

Nesta fase, a arquitetura do sistema foi reorganizada face à fase anterior. A principal alteração estrutural foi a separação da lógica de *parsing* XML da classe `World`, que na fase 1 acumulava responsabilidades tanto de armazenamento da cena como de leitura do ficheiro de configuração.

Esta separação originou uma nova classe, `SceneParser`, que centraliza toda a lógica de interpretação do XML. A classe `World` passou a ser uma estrutura de dados pura, limitando-se a agregar a janela (`Window`), a câmara (`Camera`) e o grupo raiz da cena (`Group`).

O diagrama acima ilustra como o `SceneParser` medeia a relação entre o ficheiro XML e a estrutura de dados em memória. Internamente, o `SceneParser` utiliza a biblioteca `tinycl2` para navegar na árvore XML, delegando a construção dos objetos às respetivas classes.

2.1. Classe `SceneParser`

A classe `SceneParser` expõe um único método público estático:

```
loadWorld(filename : string, world : World) : bool
```

Este método coordena o carregamento completo da cena, invocando internamente quatro funções auxiliares privadas: `parseWindow`, `parseCamera`, `parseModels` e `parseGroup`. Cada uma destas funções recebe o elemento XML correspondente e preenche a estrutura de dados adequada.

A função `parseGroup` é recursiva: ao encontrar elementos `<group>` filhos dentro de um grupo, invoca-se a si própria para construir o subgrupo, que é depois adicionado ao grupo pai. Esta recursividade natural reflete a estrutura em árvore do XML.

A função `parseTransform` itera pelos filhos do elemento `<transform>`, identificando cada transformação pelo seu nome (`translate`, `rotate` ou `scale`) e instanciando o objeto concreto correspondente, que é adicionado ao grupo através de um `unique_ptr<Transformation>`.

2.2. Hierarquia Transformation

Para representar as transformações geométricas de forma extensível, foi definida uma hierarquia de classes com base numa classe abstrata `Transformation`:

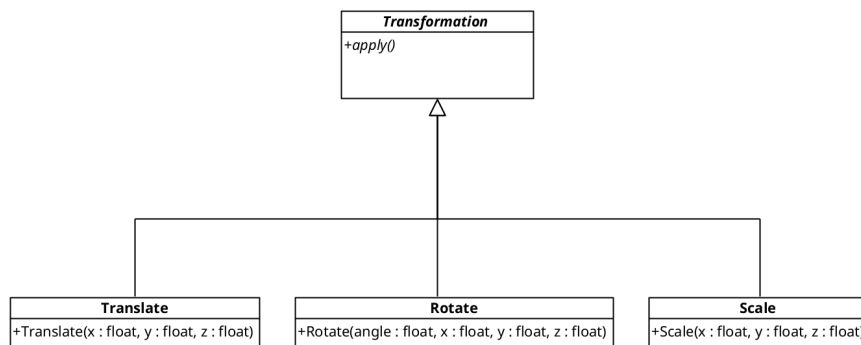


Figura 1: Hierarquia Transformation

A classe base declara um único método virtual puro `apply()`, que cada subclasse implementa recorrendo diretamente às funções da *pipeline* OpenGL:

- `Translate::apply()` invoca `glTranslatef(x, y, z)`
- `Rotate::apply()` invoca `glRotatef(angle, x, y, z)`
- `Scale::apply()` invoca `glScalef(x, y, z)`

Esta abordagem permite que o grupo itere sobre uma lista heterogénea de transformações e as aplique em sequência, sem necessidade de conhecer o tipo concreto de cada uma. A ordem de inserção na lista é respeitada, o que é determinante para o resultado final, dado que as transformações geométricas não são, em geral, comutativas.

2.3. Classe Group

A classe `Group` foi reformulada para acomodar três coleções distintas:

- lista de transformações
- lista de modelos
- lista de sub-grupos

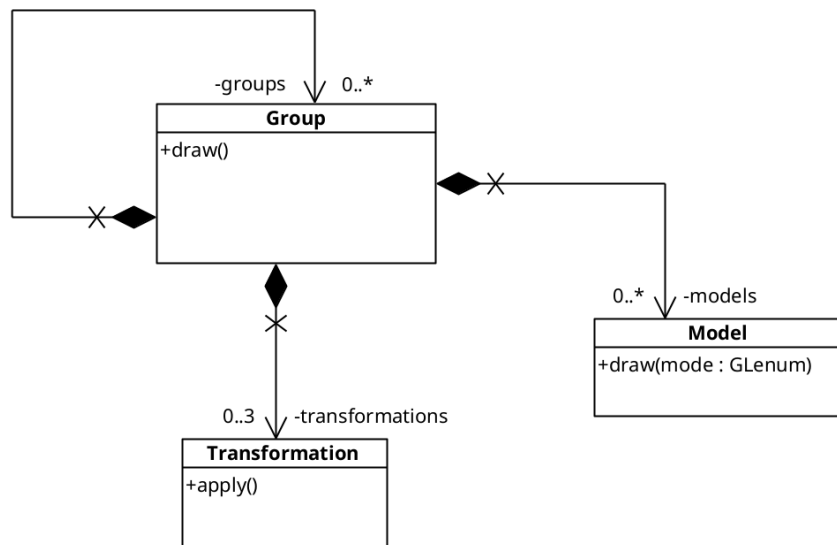


Figura 2: Estrutura da classe Group

As transformações são guardadas como `std::vector<std::unique_ptr<Transformation>>`, o que permite armazenar objetos polimórficos de forma eficiente e com gestão automática de memória. Os subgrupos são armazenados como `std::vector<Group>`, suportando aninhamento arbitrário.

O método `draw()` do grupo segue a seguinte sequência:

1. Chamar `glPushMatrix()` para salvar o estado da matriz de transformação atual.
2. Aplicar, em ordem, todas as transformações do grupo (`t→apply()` para cada `t`).
3. Desenhar todos os modelos do grupo.
4. Recursivamente chamar `draw()` em cada subgrupo filho.
5. Chamar `glPopMatrix()` para restaurar a matriz ao estado anterior.

Este mecanismo de *push/pop* garante que as transformações de um grupo não se propagam para os seus irmãos, mas propagam-se corretamente para os seus filhos, implementando assim a herança hierárquica de transformações.

3. Transformações Geométricas

Nesta secção descrevem-se as três transformações geométricas suportadas, a sua representação matemática e o seu efeito na *pipeline* de renderização do OpenGL.

3.1. Translação

A translação desloca um objeto no espaço tridimensional ao longo dos eixos x , y e z . Matematicamente, corresponde à adição de um vetor de deslocamento $\mathbf{t} = (t_x, t_y, t_z)$ às coordenadas de cada vértice:

$$\mathbf{P}' = \mathbf{P} + \mathbf{t} = (x + t_x, \quad y + t_y, \quad z + t_z) \quad (1)$$

Em termos de álgebra linear com coordenadas homogêneas, a translação é representada pela matriz 4×4 :

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2)$$

No contexto da Seção 4 (Demo Scene), a translação é frequentemente usada para posicionar objetos relativamente ao seu grupo pai. Por exemplo, um planeta é movido para a sua órbita em relação ao Sol, que está na origem do grupo raiz.

3.2. Rotação

A rotação de ângulo α em torno de um eixo arbitrário $\hat{u} = (u_x, u_y, u_z)$ é descrita pela fórmula de Rodrigues. Os casos particulares mais comuns são a rotação em torno dos eixos canónicos. Por exemplo, a rotação em torno do eixo y (usado para posicionar planetas na sua órbita ao redor do Sol) corresponde a:

$$R_y(\alpha) = \begin{pmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (3)$$

Na Demo Scene (Seção 4) desta fase, a rotação é aplicada antes da translação para colocar os planetas em posições angulares distintas ao redor do Sol. A ordem Rotação $\rightarrow$ Translação $\rightarrow$ Escala é significativa: ao rodar primeiro e depois movimentar, o objeto é deslocado radialmente numa direção determinada pelo ângulo de rotação, produzindo o efeito de órbita estática.

3.3. Escala

A escala altera as dimensões de um objeto por fatores independentes em cada eixo:

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (4)$$

Quando $s_x = s_y = s_z = s$, a escala é uniforme e preserva a forma do objeto. No sistema solar, a escala uniforme é usada para representar o tamanho relativo dos planetas em relação ao Sol.

4. Demo Scene

A *demo scene* desta fase é um modelo estático do sistema solar, composto pelo Sol, oito planetas e dois satélites naturais (a Lua da Terra e uma lua de Júpiter), bem como o anel de Saturno.

4.1. Estrutura Hierárquica

A cena é organizada numa árvore de grupos. O grupo raiz representa o Sol – uma esfera com escala 6, centrada na origem. Os planetas são subgrupos diretos do grupo raiz; a sua posição orbital é definida pela composição Rotação → Translação → Escala aplicada a cada subgrupo.

A estrutura hierárquica segue o seguinte padrão para cada planeta:

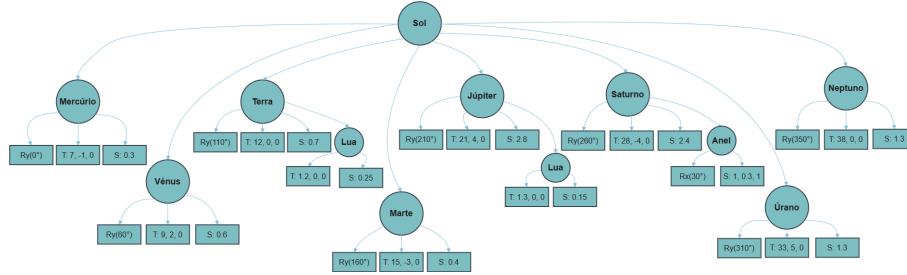


Figura 3: Estrutura hierárquica

A notação $R_y(\alpha)$ indica rotação em torno do eixo y de ângulo α , $T(x, y, z)$ indica translação e $S(s)$ indica escala uniforme de fator s .

4.2. Herança de Transformações

A lua da Terra é um subgrupo do grupo da Terra. Quando a Terra é posicionada pela sequência $R_{y(110^\circ)} \rightarrow T(12, 0, 0) \rightarrow S(0.7)$, a lua herda todas essas transformações, acumuladas na matriz do OpenGL. A translação local da lua de $T(1.2, 0, 0)$ é então aplicada no espaço já transformado do planeta, colocando-a a uma distância de 1.2 unidades do centro da Terra (após escala). O mesmo princípio aplica-se ao anel de Saturno, que herda a posição e escala do planeta.

O mecanismo de `glPushMatrix` / `glPopMatrix` garante que a transformação da Terra (e da sua lua) não afeta os restantes planetas na hierarquia, isolando o estado da matriz a cada nível da árvore.

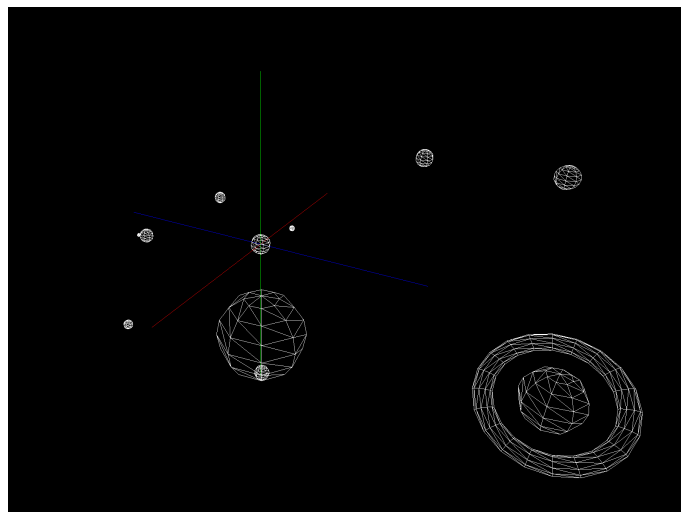


Figura 4: Sistema Solar no OpenGL

5. Controlo da Câmara

O sistema de controlo de câmara introduzido nesta fase utiliza o rato como dispositivo de interação, em substituição das teclas direcionais usadas na fase anterior. A câmara orbita em torno da origem do mundo em coordenadas esféricas, definidas pelo raio r , o ângulo horizontal α e o ângulo vertical β .

A posição cartesiana da câmara é recalculada a cada *frame* pelas equações:

$$x = r \times \cos(\beta) \times \sin(\alpha) \quad (5)$$

$$y = r \times \sin(\beta) \quad (6)$$

$$z = r \times \cos(\beta) \times \cos(\alpha) \quad (7)$$

A interação é gerida através de três *callbacks* GLUT:

- **glutMouseFunc**: deteta o botão premido e regista a posição inicial do cursor. O botão esquerdo ativa o modo de rotação orbital (*tracking* = 1) e o botão direito ativa o modo de *zoom* (*tracking* = 2).
- **glutMotionFunc**: enquanto o botão está premido, calcula o delta de deslocamento do cursor $\Delta x = x_{\text{atual}} - x_{\text{inicio}}$ e $\Delta y = y_{\text{atual}} - y_{\text{inicio}}$. No modo de rotação, estes deltas são somados aos ângulos α e β respetivamente, produzindo rotação orbital. No modo de *zoom*, o delta vertical é subtraído ao raio r .

O ângulo β é limitado ao intervalo $[-85^\circ, 85^\circ]$ para evitar a passagem pelo polo, que causaria inversão do vetor *up* e consequente comportamento indefinido na câmara. O raio r é limitado a um mínimo de 3 unidades para evitar que a câmara penetre os modelos.

6. Conclusão

A segunda fase do projeto permitiu consolidar os conceitos fundamentais de cenas hierárquicas e transformações geométricas em OpenGL.

A reestruturação arquitetural — com a separação entre *SceneParser*, *World*, *Group* e a hierarquia *Transformation* — resultou num sistema mais modular e preparado para as extensões previstas nas fases seguintes, nomeadamente a introdução de transformações animadas (curvas de Catmull-Rom e rotações temporais) e a migração do *rendering* para VBOs.

A construção do sistema solar estático demonstrou a correta implementação da herança de transformações: planetas com luas e anéis posicionam-se automaticamente no espaço correto graças ao mecanismo de *glPushMatrix* / *glPopMatrix*, sem necessidade de calcular explicitamente as coordenadas absolutas de cada objeto filho.

O controlo de câmara por rato, desenvolvido como funcionalidade extra, melhora significativamente a exploração interativa da cena, tornando a navegação mais intuitiva do que a abordagem por teclas da fase anterior.